

Real-Time Tetris Game Implementation on a MIPS-Based Embedded System

[Arash Zakipour] and [Jacob Polanco Johansson]

Objective and Requirements

- **Display Management:** An easier way to use the display should be found
- **Shapes:** Several different shapes should have been made
- **Spawing new block:** After a block reaches bottom it should spawn a new block
- **Can move left/right/down:** A block should not move to left or right if there is something there, also should remain stationary when a block is under it.
- **Saving block:** After blocks become stationary we need to have something to memorize their locations so they are on as we spawn new blocks.
- **Complete Row:** When a row is complete, the row should turn off, the rows above should come down, the score should increase
- **Game Over:** If vertically all elements of the grid is filled, the game should be over

Solution

- **Display Management:** We have used the set pixel function to have as input x and y like a cartesian map and
- **Shapes:** We have defined our own types for the shapes. Where one value is a nxn grid containing the instructions for how the program will display the shape. And the other the value for the length and height. As the shape is moving around the screen it is seen as its own entity. But the second it is unable to keep going downwards, the shape gets stored into the grid. This is done by taking the shapes position in the grid, which is set to the top right corner of the shape. Then going through the shape matrix. If the value in the matrix is set to 1, it will be set to 1 on the grid. And a new one appears at the top of the screen.
- **Spawing new shape:** When a shape is unable to keep going downwards, the shape gets stored into the grid. And a new one appears at the top of the screen. The program has a seed and will choose a random number based on that seed. Which is then sued to access a certain shape from the shapes array.
- **Can move left/right/down:** It grabs the current position of our shape and looks at 1 left, 1 right and 1 down respectively block in grid to check if it is 1 or 0. If it 0 it can go to those positions and it returns true, otherwise False.
- **Saving block:** We have made a grid of our cartesian map that we constantly render and we add the blocks to their correponding x ,y position on the grid as soon as they become stationary, ie when can move down is false

- **Complete Row:** A function that constantly loops through our grid after a falling blocks happened and searches for a completed row and when it happens,
- **Game Over:** Game over function is constantly running throughout the labwork function and it looks for any columns that is completely filled over our grid, more accurately it counts how many 1s there is in a column and if it is pass 127 it makes game over.

Verification

Verification was performed in three stages:

1. **Actual Testing:** For most things such as collisions, saving block, row complete and etc we directly tested them by viewing them in the displaer. However at times in order to ease and reduce time spending we did have to make testing functions that facilitated the task of tasting certain aspect as we explain in the following section.
2. **Testing Functions:** We have made several functions that directly forced a situation. For example we used test set pixel to check if our set pixel function actually worked by choosing several different points to figure out how the x and y axis works on the device and improve as go. Another example was us feeding several rows of the grid to 1 by using test grid function to first ensure if row complete happens. We firstly made it so that whenever a row is completed it turns the whole grid to one as in every thing lights up. After we made sure when the condition is met the function is called, we changed the function to have the bottom 3 rows on and some blocks above it to see whether the blocks come down.

Contributions

We both worked on most of the things together, but the following specifies what each of us mainly worked on.

- **[Arash Zakipour]:** Worked on making the display into a cartesian plane, making a memory of dropped blocks, testing functions and clear row score.
- **[Jacob Polanco Johansson]:** Worked with can move left/right/down, rotations, clear row increase score, game start and over. As well as showing score.

Reflections

This project was without the doubt challenging. The individual components may themselves not have been too difficult to implment however the interationcs with one another and the actual result on the display required a lot of debugging and fixings which helped the two of us imprpove and finding steps on how to debug a program.